COSC 423/523: Introduction to Artificial Intelligence

Reinforcement Learning



THE UNIVERSITY OF TENNESSEE KNOXVILLE

Credits to UC Berkeley CS188 (Introduction to Artificial Intelligence), with thanks to Professor Pieter Abbeel for sharing the materials.

Exam 2 – Oct 30 (3:00 pm-3:50 pm)

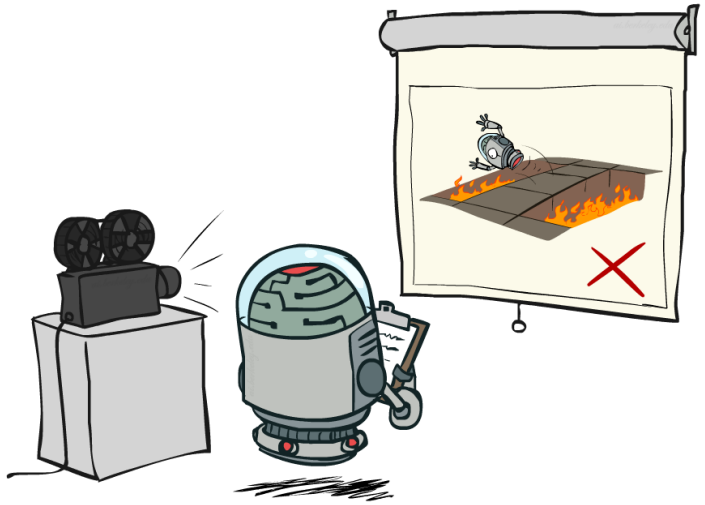
- **Topics**
  - **Lectures 16-22:** State values and Q-values; Bellman equations; Value iteration; Policy evaluation and policy iteration; Model-based and model-free learning; Monte Carlo evaluation; Temporal Difference (TD) learning; Q-learning; Approximate Q-learning

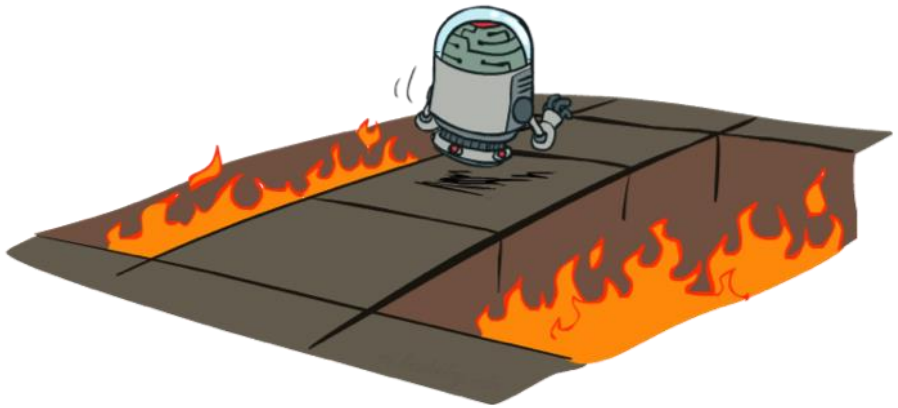
- **Format & Policies**
  - **Closed-book, closed-notes, one blank sheet of paper is allowed for scratch work only**
  - **Complete the exam on Canvas using your own laptop**
  - **20 multiple-choice questions, 1 bonus question**
    - 5 points for each multiple-choice question
    - Scores are capped at 100 points

- **Not Emphasized**
  - Long derivations or proofs; Memorizing formulas without understanding; Programming

Today

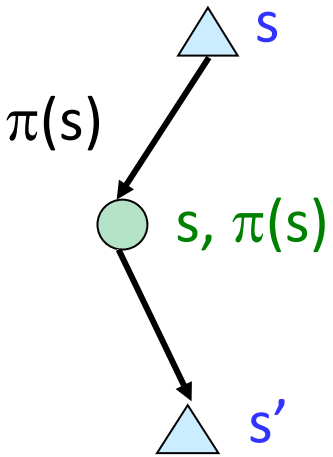
- Model-Based Learning
- Model-Free Learning
  - Passive Reinforcement Learning
    - Direct Evaluation (Monte Carlo Evaluation)
    - Temporal Difference Learning
  - Active Reinforcement Learning
    - Q-Learning





Recap: Temporal Difference Learning

- Big idea: learn from every experience!
  - Update V(s) each time we experience a transition (s, a, s', r)
  - Likely outcomes s' will contribute updates more often



- Temporal difference learning of values
  - Policy still fixed, still doing evaluation!
  - Move values toward value of whatever successor occurs: running average

Sample of V(s): $sample = R(s, \pi(s), s') + \gamma V^{\pi}(s')$

Update to V(s): $V^{\pi}(s) \leftarrow (1-\alpha)V^{\pi}(s) + (\alpha)sample$

Same update: $V^{\pi}(s) \leftarrow V^{\pi}(s) + \alpha(sample - V^{\pi}(s))$

Recap: Q-Learning

- Q-Learning: sample-based Q-value iteration

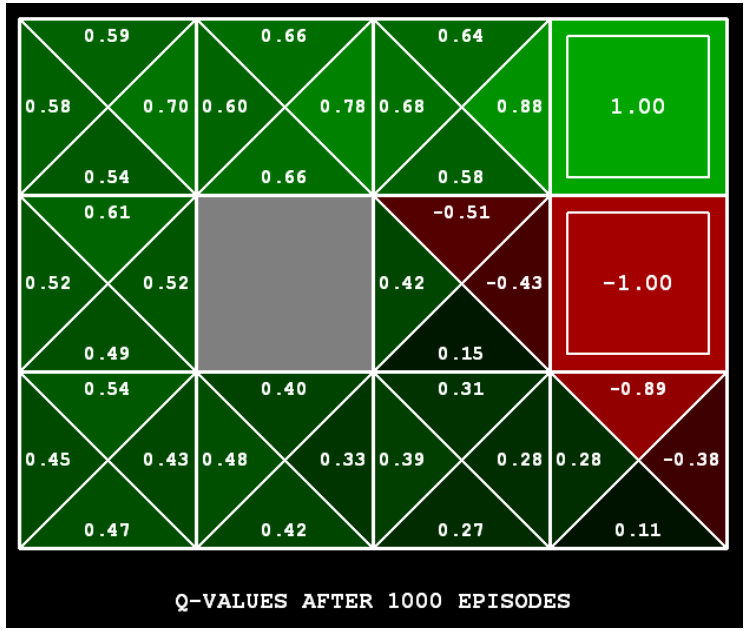
$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]$$

- Learn Q(s,a) values as you go
  - Receive a sample (s,a,r,s')
  - Consider your old estimate: $Q(s,a)$
  - Consider your new sample estimate:

$$sample = R(s,a,s') + \gamma \max_{a'} Q(s',a')$$

  - Incorporate the new estimate into a running average:

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + (\alpha)\,[sample]$$

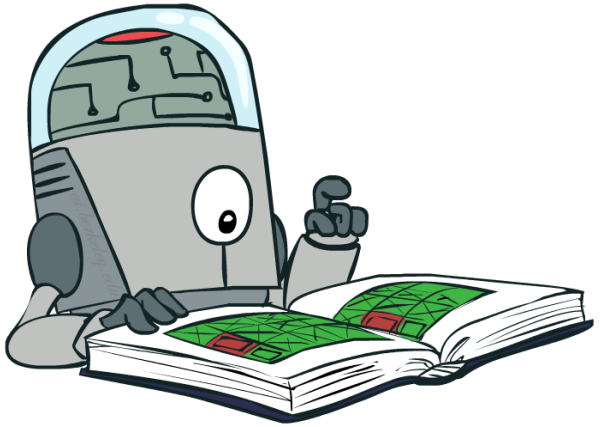


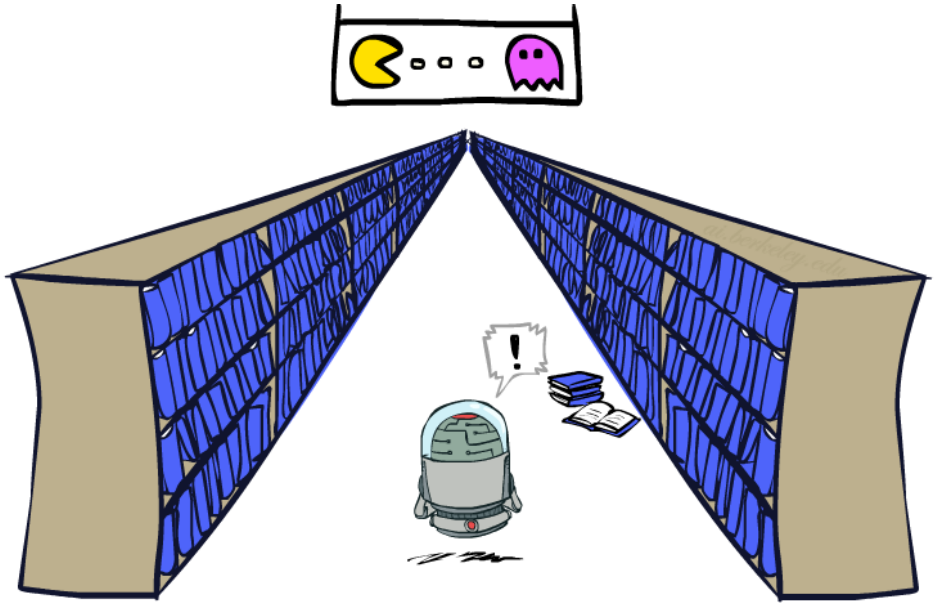
Generalizing Across States

- Basic Q-Learning keeps a table of all q-values

- In realistic situations, we cannot possibly learn about every single state!
  - Too many states to visit them all in training
  - Too many states to hold the q-tables in memory

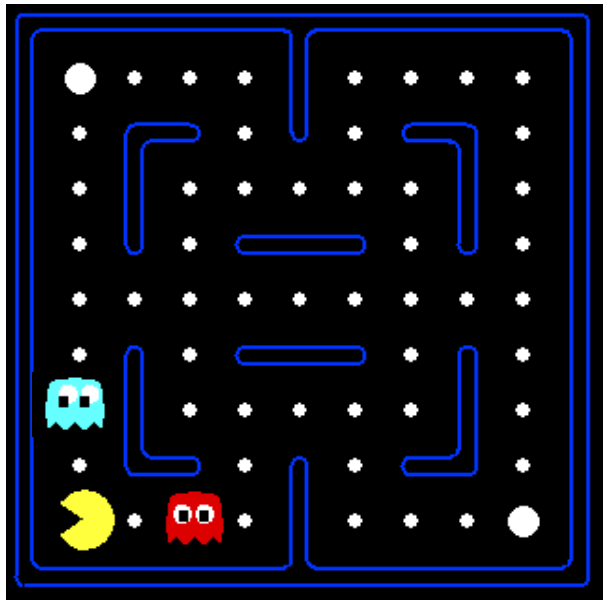
- Instead, we want to generalize:
  - Learn about some small number of training states from experience
  - Generalize that experience to new, similar situations
  - This is a fundamental idea in machine learning, and we'll see it over and over again



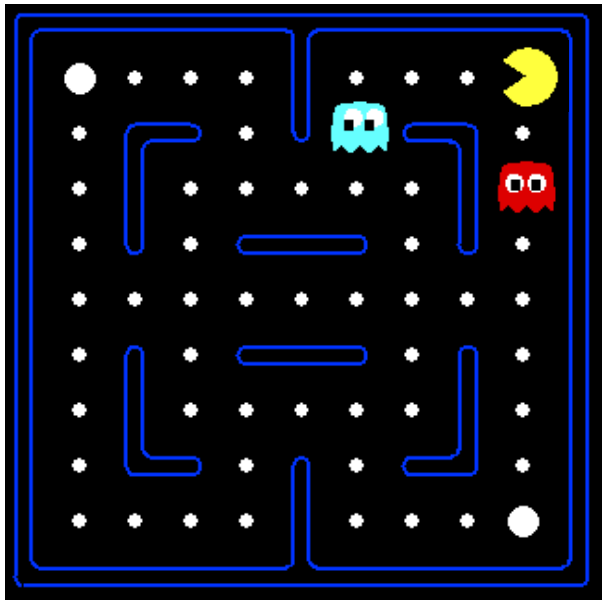


Example: Pacman

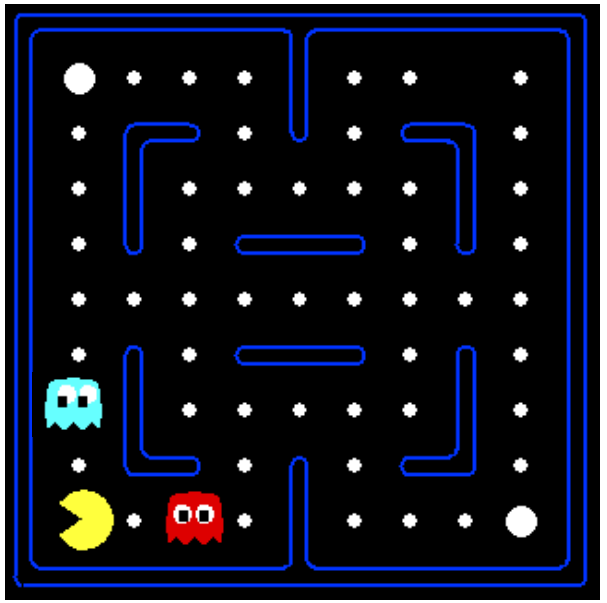
Let's say we discover through experience that this state is bad:



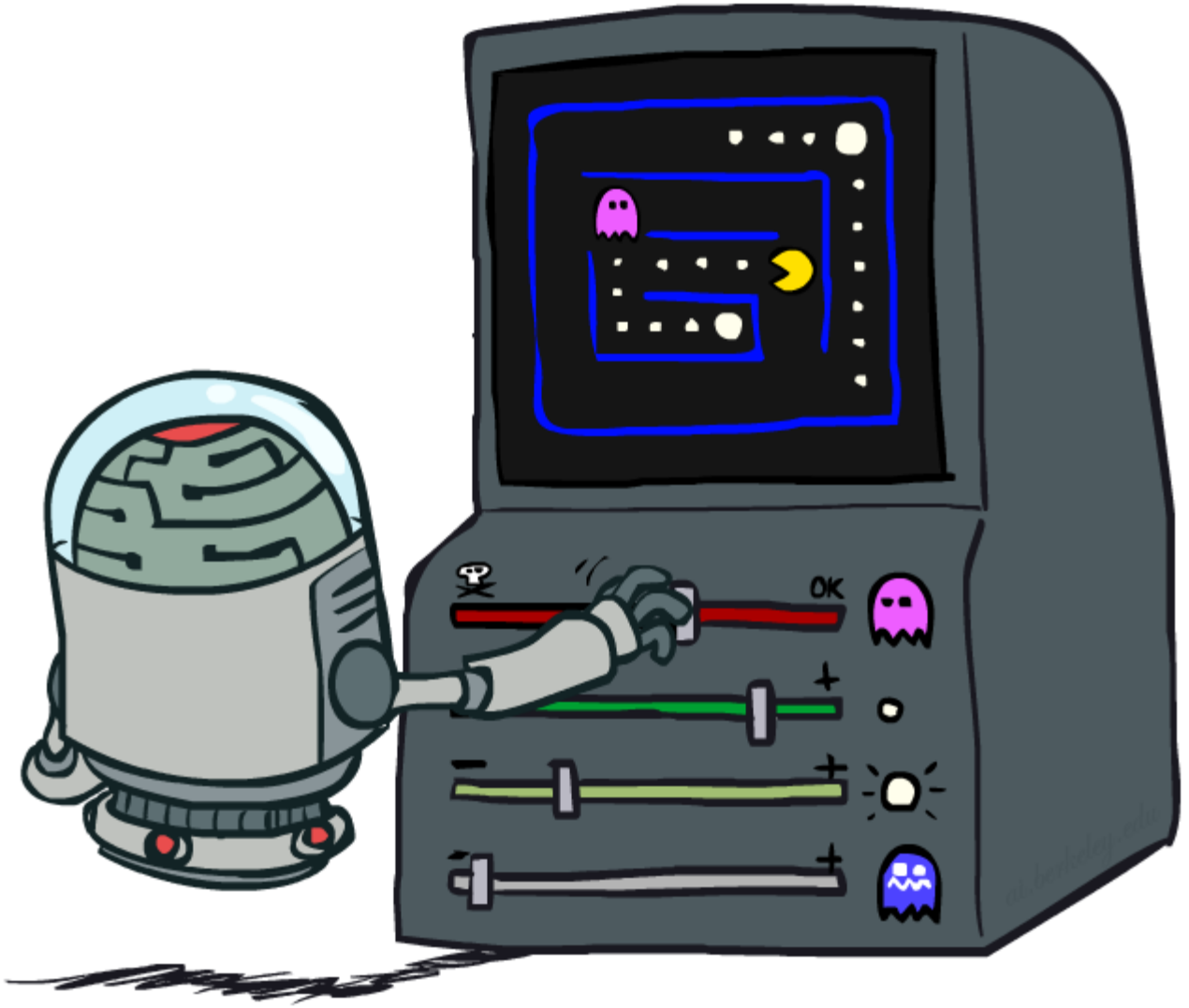
In naïve q-learning, we know nothing about this state:



Or even this one!

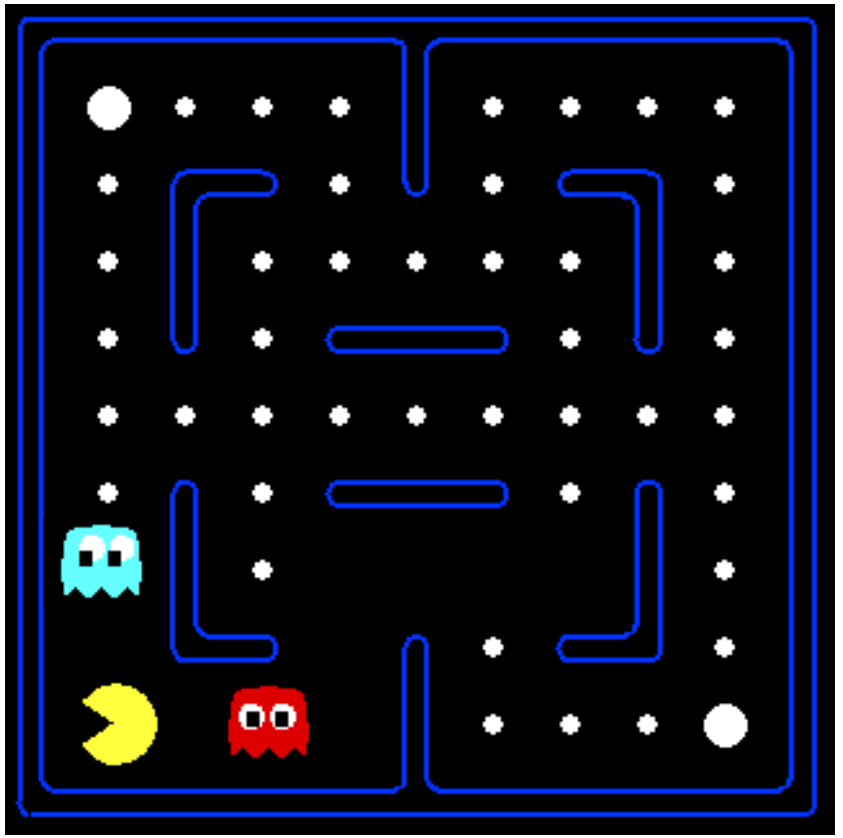


Approximate Q-Learning

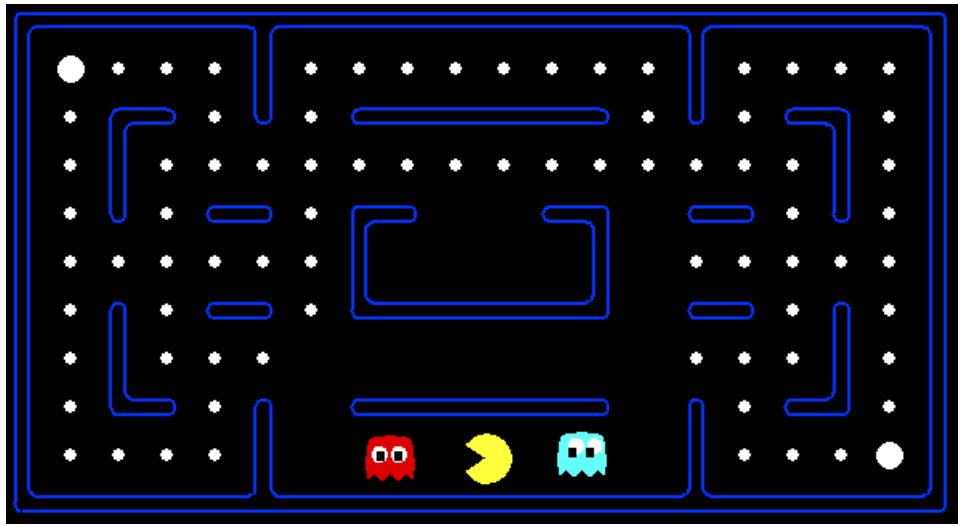


Feature-Based Representations

- Solution: describe a state using a vector of features (properties)
  - Features are functions from states to real numbers (often 0/1) that capture important properties of the state
  - Example features:
    - Distance to closest ghost
    - Distance to closest dot
    - Number of ghosts
    - Is Pacman in a tunnel? (0/1)
    - …… etc.
  - Can also describe a q-state (s, a) with features (e.g. action moves closer to food)



Evaluation Functions



- minDist: minimum distance to active ghosts (larger = safer)
- escapeR: number of safe directions Pacman can move within 1–2 steps (more exits = safer, distinguishes “being chased” vs “being trapped”)
- Bonus: positive reward when ghosts are scared, encouraging Pacman to chase them after eating a power pellet

$$Eval(s) = score(s) - a \cdot dist(food) + b \cdot minDist + c \cdot escapeR + d \cdot Bonus$$

Linear Value Functions

- Using a feature representation, we can write a q function (or value function) for any state using a few weights:

$$V(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

$$Q(s,a) = w_1 f_1(s,a) + w_2 f_2(s,a) + \ldots + w_n f_n(s,a)$$

- Advantage: our experience is summed up in a few powerful numbers

- Disadvantage: states may share features but actually be very different in value!

Approximate Q-Learning

$$Q(s,a) = w_1 f_1(s,a) + w_2 f_2(s,a) + \ldots + w_n f_n(s,a)$$

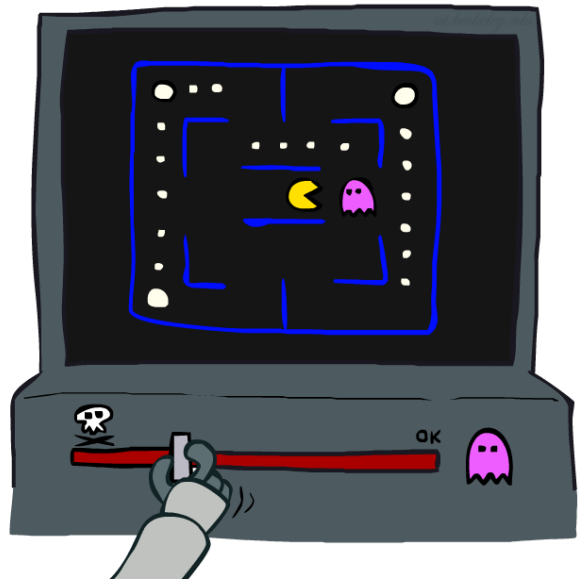
- Q-learning with linear Q-functions:

$$\text{transition} = (s, a, r, s')$$

$$\text{difference} = \left[r + \gamma \max_{a'} Q(s', a')\right] - Q(s,a)$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\text{difference}] \qquad \text{Exact Q's}$$

$$w_i \leftarrow w_i + \alpha\,[\text{difference}]\, f_i(s,a) \qquad \text{Approximate Q's}$$



- Intuitive interpretation:
  - Adjust weights of active features
  - E.g., if something unexpectedly bad happens, blame the features that were on: disprefer all states with that state's features

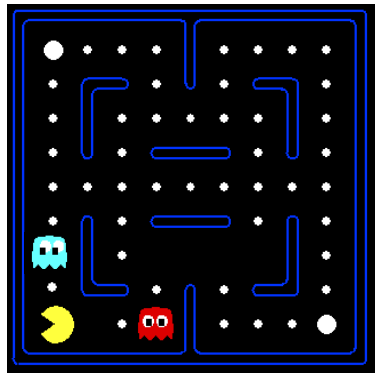
- Formal justification: online least squares
  - implemented with SGD on squared error

Example: Q-Pacman

$$Q(s,a) = 4.0 f_{DOT}(s,a) - 1.0 f_{GST}(s,a)$$

f_{DOT}: distance to nearest dot;
f_{GST} :distance to nearest ghost

s

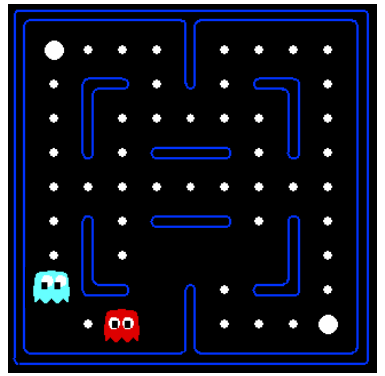


$$f_{DOT}(s, \text{NORTH}) = 0.5$$

$$f_{GST}(s, \text{NORTH}) = 1.0$$

$$a = \text{NORTH}$$
$$r = -500$$

s'



$$Q(s, \text{NORTH}) = +1$$

$$Q(s', \cdot) = 0$$

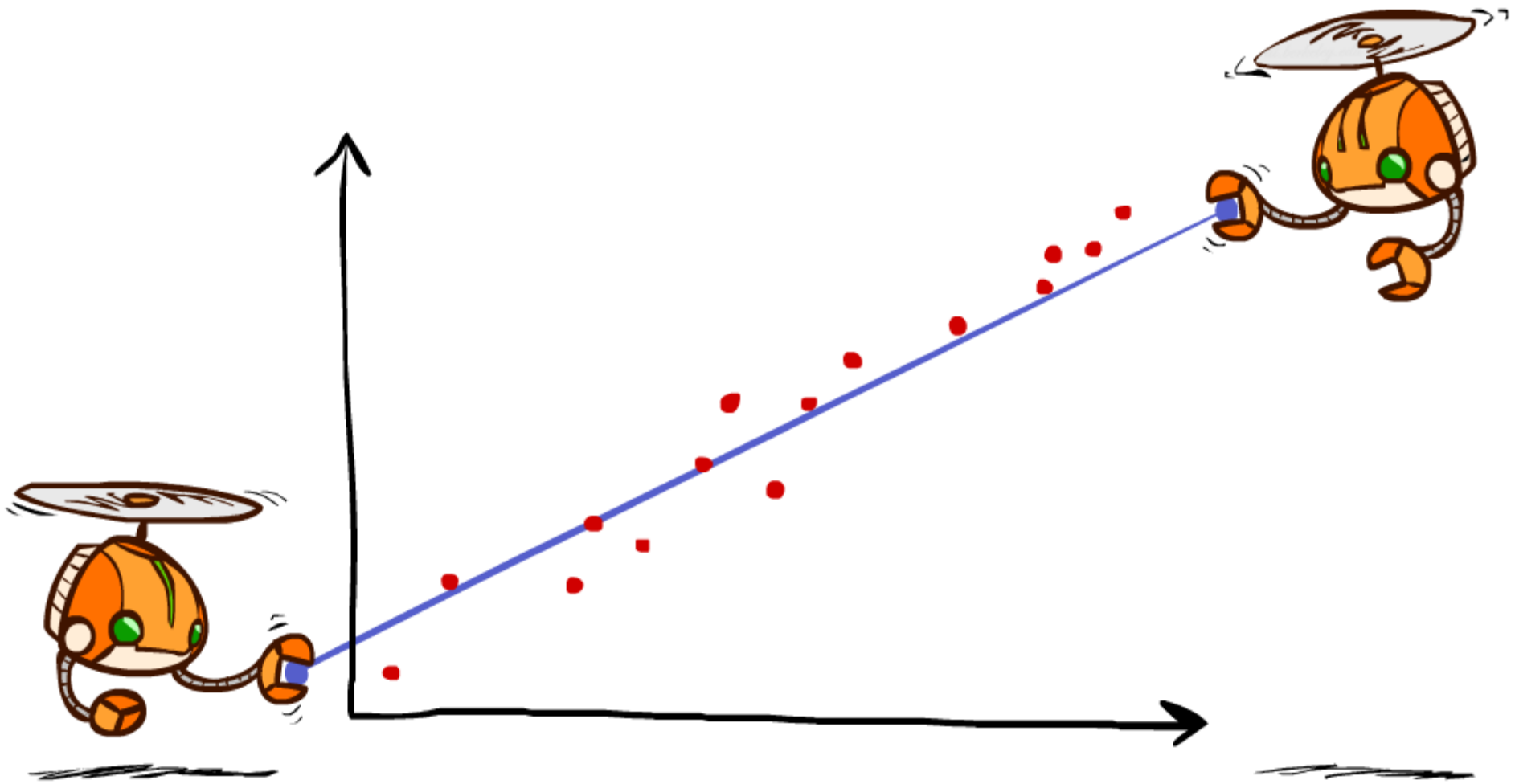
$$r + \gamma \max_{a'} Q(s', a') = -500 + 0$$

$$\text{difference} = -501$$

$$w_{DOT} \leftarrow 4.0 + \alpha\,[-501]\,0.5$$
$$w_{GST} \leftarrow -1.0 + \alpha\,[-501]\,1.0$$

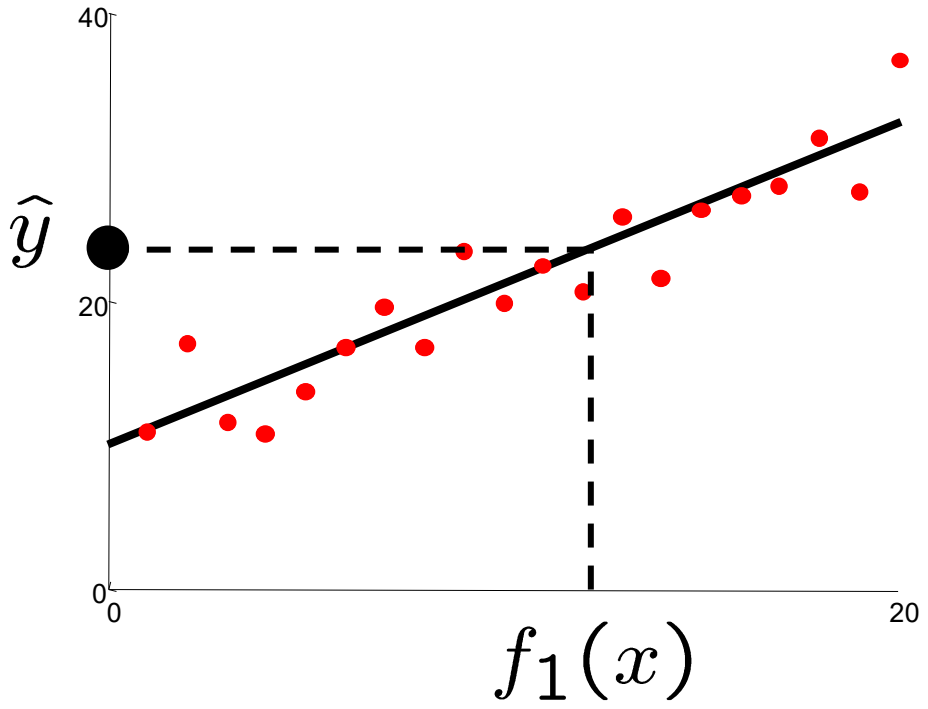
$$Q(s,a) = 3.0 f_{DOT}(s,a) - 3.0 f_{GST}(s,a)$$

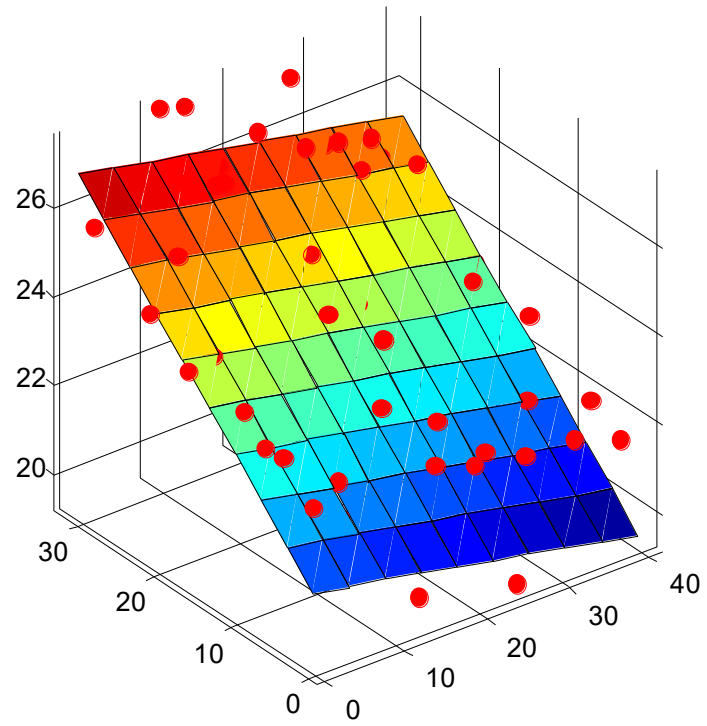
Q-Learning and Least Squares*



*: out of scope

Linear Approximation: Regression*





Prediction:

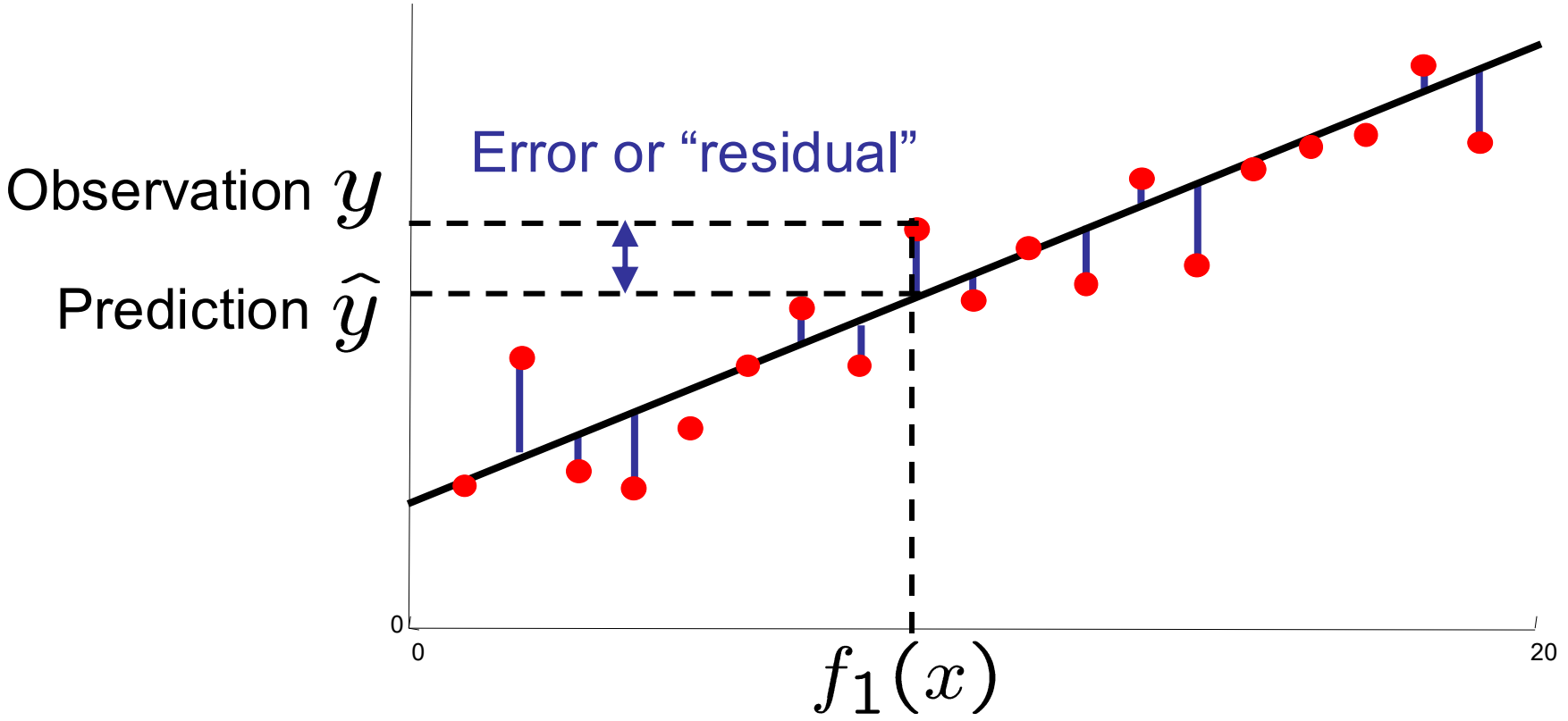
$$\widehat{y} = w_0 + w_1 f_1(x)$$

Prediction:

$$\widehat{y}_i = w_0 + w_1 f_1(x) + w_2 f_2(x)$$

Optimization: Least Squares*

$$\text{total error} = \sum_i (y_i - \hat{y}_i)^2 = \sum_i \left(y_i - \sum_k w_k f_k(x_i) \right)^2$$



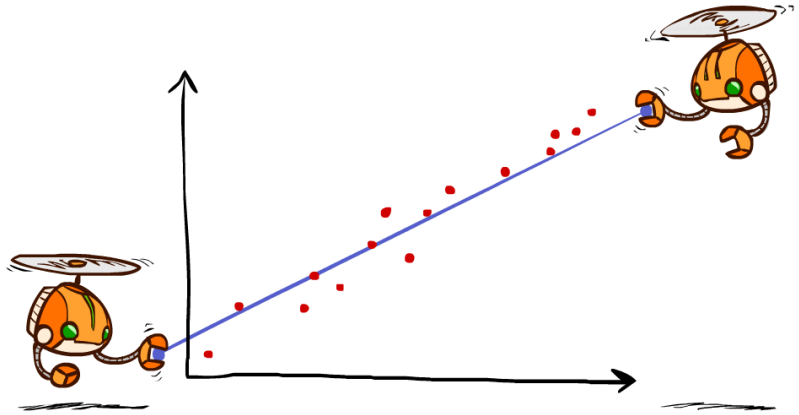
Minimizing Error*

Imagine we had only one point x, with features f(x), target value y, and weights w:

$$\text{error}(w) = \frac{1}{2}\left(y - \sum_k w_k f_k(x)\right)^2$$

$$\frac{\partial\ \text{error}(w)}{\partial w_m} = -\left(y - \sum_k w_k f_k(x)\right) f_m(x)$$

$$w_m \leftarrow w_m + \alpha\left(y - \sum_k w_k f_k(x)\right) f_m(x)$$



Approximate q update explained:

$$w_m \leftarrow w_m + \alpha\left[r + \gamma \max_a Q(s', a') - Q(s, a)\right] f_m(s, a)$$

"target" "prediction"

RL in Action*

- Pong: Learning from Pixels:
  https://www.youtube.com/watch?v=vXtfdGphr3c

- AlphaGo: Learning through Self-Play:
  https://www.youtube.com/watch?v=4PyWLgrt7YY

- ChatGPT: Learning from Human Feedback:
  https://www.youtube.com/watch?v=WMmGzx-jWvs